

SISTEMA DE MONITORAMENTO DE RISCO EM TEMPO REAL

**UTILIZANDO VISÃO COMPUTACIONAL E COMPUTAÇÃO DE
BORDA**

Relatório Técnico

Londrina, Dezembro de 2025

1. Introdução

O avanço das técnicas de visão computacional e aprendizado profundo tem possibilitado o desenvolvimento de sistemas capazes de interpretar informações visuais de forma automática e em tempo real. Em ambientes industriais, tais tecnologias são amplamente empregadas para aumentar a segurança operacional, reduzir riscos e auxiliar na tomada de decisões relacionadas à prevenção de acidentes.

Nesse contexto, o monitoramento contínuo de atividades potencialmente perigosas representa um desafio relevante, especialmente em cenários onde a supervisão humana constante é inviável ou sujeita a falhas. Sistemas automatizados de detecção de objetos permitem identificar situações de risco de forma proativa, contribuindo para a mitigação de acidentes e para o cumprimento de normas de segurança.

Este relatório apresenta a documentação técnica de uma Prova de Conceito (PoC) denominada edge-risk-monitor, um sistema de monitoramento de risco baseado em visão computacional, capaz de realizar inferência em tempo real utilizando computação de borda (edge computing). Como objeto de risco, foi adotado um mouse de computador, utilizado como substituto conceitual de um item perigoso, permitindo a validação do sistema sem exposição a riscos reais.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema de monitoramento de risco em tempo real baseado em visão computacional, capaz de identificar automaticamente a presença de um objeto de risco em um fluxo contínuo de vídeo e registrar eventos associados à detecção.

2.2 Objetivos Específicos

- Implementar um sistema de captura contínua de imagens por meio de webcam.
- Realizar inferência local em tempo real utilizando um modelo de detecção de objetos.
- Aplicar um mecanismo de debounce temporal para reduzir falsos positivos.
- Gerar evidências visuais associadas às detecções confirmadas.
- Enviar eventos estruturados para um backend desacoplado via API HTTP.
- Garantir organização, modularidade e rastreabilidade do código-fonte.

3. Visão Geral do Sistema

O sistema edge-risk-monitor foi projetado para operar de forma independente, executando todas as etapas de inferência localmente. A arquitetura adotada segue o paradigma de computação de borda, no qual o processamento ocorre próximo à fonte dos dados, reduzindo latência e dependência de serviços externos.

A solução é composta por módulos independentes, cada um com responsabilidade bem definida, sendo o arquivo main.py responsável exclusivamente pela orquestração do fluxo de execução.

4. Arquitetura do Sistema

- A arquitetura do sistema foi organizada de forma modular, visando facilitar manutenção, testes e evolução futura. Os principais componentes são:
- Webcam: responsável pela captura contínua de frames.
- Detector: executa inferência utilizando um modelo YOLO treinado.
- EvidenceSaver: persiste evidências visuais das detecções.
- EventSender: envia eventos e evidências para um backend via API.
- Logging Global: centraliza a configuração e padronização dos logs.
- Main: coordena o fluxo entre os componentes.

Essa separação garante baixo acoplamento e alta coesão entre os módulos.

5. Metodologia de Desenvolvimento

5.1 Captura de Vídeo

A captura de imagens é realizada por meio da biblioteca OpenCV, utilizando uma webcam conectada ao sistema. O módulo responsável abstrai completamente o acesso ao dispositivo físico, fornecendo frames no formato adequado para inferência.

5.2 Inferência em Tempo Real

A detecção do objeto de risco é realizada por um modelo YOLO treinado previamente. O modelo é carregado durante a inicialização do sistema e executa inferência frame a frame, retornando bounding boxes, classes e níveis de confiança. Apenas detecções que atendem ao limiar mínimo de confiança e à classe de interesse são consideradas válidas.

5.3 Debounce Temporal

Para evitar falsos positivos causados por ruído ou detecções esporádicas, foi implementado um mecanismo de debounce temporal. O risco é confirmado apenas após a detecção consecutiva do objeto em um número mínimo de frames. Da mesma forma, o estado de risco é limpo somente após um número definido de frames sem detecção.

5.4 Geração de Evidências

Uma vez confirmado o risco, o sistema salva uma imagem do frame correspondente, que serve como evidência visual do evento. As evidências são armazenadas localmente em um diretório específico, garantindo rastreabilidade e auditoria posterior.

5.5 Envio de Eventos

Após a geração da evidência, o sistema envia um evento estruturado para um backend desacoplado por meio de uma requisição HTTP POST. O evento contém informações como endereço MAC do dispositivo, data e hora da ocorrência, classe detectada e a imagem de evidência.

6. Registro e Monitoramento

O sistema utiliza o módulo de logging padrão do Python, com configuração centralizada, registrando eventos relevantes em tempo real. Os logs incluem informações de inicialização, detecção, envio de eventos, falhas e encerramento da aplicação, sendo gravados tanto no terminal quanto em arquivo.

7. Resultados Obtidos

Durante os testes realizados, o sistema demonstrou capacidade de detectar o objeto de risco em tempo real, confirmando corretamente os eventos após o debounce temporal e evitando múltiplos envios para a mesma ocorrência. As evidências visuais foram geradas conforme esperado, e os eventos foram enviados com sucesso para o backend configurado. Os logs gerados permitiram acompanhar de forma clara todo o ciclo de vida do sistema, desde a captura dos frames até o envio final dos eventos.

8. Limitações

Embora funcional, o sistema desenvolvido apresenta algumas limitações inerentes à sua natureza de Prova de Conceito (PoC), as quais devem ser consideradas na análise dos resultados obtidos.

Entre as principais limitações, destacam-se:

- O sistema oferece suporte a apenas uma câmera por instância, não contemplando cenários com múltiplos pontos de monitoramento simultâneos.
- A detecção está restrita a um único tipo de objeto de risco, definido previamente durante o treinamento do modelo.
- O desempenho da detecção é dependente das condições de iluminação e do posicionamento da câmera, fatores que podem influenciar diretamente a precisão do modelo.
- O envio dos eventos para o backend não contempla mecanismos de autenticação ou criptografia, uma vez que tais aspectos não fazem parte do escopo desta PoC.

9. Conclusão

O desenvolvimento do edge-risk-monitor demonstrou a viabilidade de um sistema de monitoramento de risco baseado em visão computacional operando em tempo real por meio de computação de borda. A arquitetura modular adotada facilitou a organização do

código e a integração entre os componentes, enquanto o uso de técnicas como debounce temporal contribuiu para a robustez do sistema.

A solução apresentada atende aos objetivos propostos e pode servir como base para sistemas mais complexos de monitoramento industrial.